Toom-Cook / Toom-3 Executive Summary

Sri Kapa
Ryan Leyhe
Anthony Wang
Sasha Arditi

Introduction:

Multiplying large numbers is a foundation of modern computer science, essential in fields like cryptography (specifically RSA encryption), large scale scientific simulations, and high performance computing. While the schoolbook method of multiplication is easy to understand, it is quite inefficient with a quadratic time complexity of $O(n^2)$. This makes it far too slow for the massive operands handled by modern systems. Toom-Cook employs a divide and conquer method to speed this up by treating numbers as polynomials.

The Toom-Cook algorithm splits operands into an arbitrary k parts, evaluating them at $2k - 1$ points. This reduces the number of sub-multiplications, yielding an asymptotic complexity of $\Theta(n^{log(2k-1)/log(k)})$. Although *k* can be any integer, the computational overhead of the interpolation phase increases as *k* grows. For this reason, Toom-3 (where *k = 3*) is the most common implementation, as it provides an optimal balance between theoretical speedup and practical efficiency for most large-scale applications.

Toom-Cook occupies a vital niche in the hierarchy of multiplication algorithms. It is the preferred choice when operands are too large for the simpler Karatsuba algorithm (when *k=2*) to remain competitive, yet have not reached the astronomical sizes required to justify the significant constant-time overhead of Fast Fourier Transforms (FFT).

Today, it remains a staple in arbitrary-precision "BigInt" libraries and secure communication protocols worldwide.

Overall Intuitive Approach / Solution:

Before diving into the mechanics of the problem, it is important to highlight once again the difference between standard multiplication and this divide and conquer solution. Standard multiplication focuses on digit by digit manipulation and therefore a quadratic time complexity, but Toom-Cook looks at the problem algebraically rather than arithmetically.

Toom-Cook, including Toom-3, treats large numbers not just as digits, but as polynomials. By representing numbers using a specific base *B,* the number can be split into *k* smaller parts, creating a polynomial of degree *k - 1.* The base is defined as $B = 2^s$, where s is the length of each chunk. For example when *k = 3,* the number 123,456 becomes $X(t) = 12t^2 + 34t + 56$

This conversion is done for both operands, leaving us with two polynomials, $X(t)$ and $Y(t)$. With this example, B would be 10^2 or 100. The final equation we are looking for then becomes $W(t) = X(t) * Y(t) = c_0 + c_1(t) + c_2(t)^2 + c_3(t)^3 + c_4(t)^4$.

1. The numbers are evaluated at $2k - 1$ distinct "magic" points.
    - In Toom-3 these would be $\{0, 1, -1, -2, \infty\}$.
    - These specific points allow us to solve the upcoming systems of equations with simple preset formulas.
        i. Ex. with $t = 1$ evaluating $X(1)$ collapses the polynomial into addition: $12 + 34 + 56 = 102$.
2. With the specific points evaluated, each corresponding points are multiplied together to give us the evaluated magic points of $W(t)$.
    - $W(-2) = X(-2) * Y(-2)$
    - Recursion is used if the evaluated points are longer than a given threshold. Instead of simply multiplying, the Toom-cook algorithm is called once more on these smaller operands.
3. With $W(t)$ evaluated for each of the magic points, we can now solve for the coefficients, c_0 to c_4
    - This is a system of equations, but since we chose the specific "magic" points, it can be simplified down to a hardcoded, optimized sequence of operations. These operations are specified in the implementation section below.
4. Lastly, with the coefficients of $W(t)$ found, we can plug in the base *B* back into the polynomial to solve for the final product.
    - With $B = 100$, we get:
      $W(100) = c_0 + c_1(100) + c_2(100)^2 + c_3(100)^3 + c_4(100)^4$

By isolating the multiplication to these specific x-coordinates, Toom-Cook reduces the number of expensive recursive multiplications compared to the schoolbook method.

Implementation:

Python:

In our Python implementation we used a generalized Toom-Cook algorithm splitting operands into an arbitrary k parts, evaluating them at $2k - 1$ points. Python is especially suited for this algorithm because it natively supports arbitrary-precision integers.

As mentioned above, in standard implementations (like Toom-3), the matrix inversion required to recover the polynomial coefficients is bypassed using hardcoded algebraic reductions evaluated at fixed points. However, our generalized k-way implementation dynamically generates evaluation points and solves the resulting linear system at runtime.

The first standard phase of the algorithm is splitting or formulating polynomials. Given two large integers X and Y in base b, we choose a splitting radix $B = b^i$ such that X and Y can be represented as polynomials of degree $k - 1$:

In our generalized implementation, the algorithm dynamically generates a sequence of small integer evaluation points, $t_i \in \{0, 1, -1, 2, -2 \ldots\}$. For example, for $k = 3$, we need exactly $2(3) - 1 = 5$ points.

For each point, the evaluation step is the same as noted above for Toom-3.

The interpolation phase is the core technical challenge of the generalized algorithm. We have a set of $2k - 1$ pointwise answers, but we need the actual coefficients of our product polynomial $c_0, c_1, c_2, \ldots)$. We find them by solving a system of linear equations using two interesting mathematical concepts.

First is the Vandermonde Matrix. A Vandermonde matrix is a specific type of grid where each row is a geometric progression of an evaluation point $(1, t, t^2, t^3, \ldots)$. Because polynomials are built on increasing powers of t, this matrix perfectly maps our evaluation points to our unknown coefficients.

Second is Gaussian Elimination, a classic algorithm used to solve systems of linear equations. Because the evaluation points change based on the dynamic parameter k, the Vandermonde matrix cannot be hardcoded. The algorithm must construct it on the fly and solve the system $V \cdot C = W$ using Gaussian Elimination. Standard floating-point Gaussian elimination will fail due to precision loss. Because the coefficients of $W(t)$ are guaranteed to be exact integers, the matrix solver must utilize exact arithmetic. In other languages, this requires writing a custom rational number class or complex LCM row scaling. In our implementation, we elegantly bypass this by utilizing Python's built-in `fractions.Fraction` module. By casting matrix operations as Fractions, Python seamlessly guarantees zero precision loss during row division.

```
# Pseudocode: Dynamic Interpolation via Vandermonde Matrix
def interpolate_coefficients(points, W_values, k):
    n = 2 * k - 1
    V = create_empty_matrix(n, n)

    # Construct the Vandermonde Matrix (1, t, t^2...)
    for i in range(n):
        for j in range(n):
            # Cast to Fraction to ensure exact arithmetic
            V[i][j] = Fraction((points[i])**j)

    # Solve V * C = W_values using exact arithmetic
    coefficients = exact_gaussian_elimination(V, W_values)

    return coefficients
```

Using our example here, we feed our Vandermonde matrix to the Gaussian solver alongside our 5 pointwise answers (like $13, 158$). Using the `Fraction` library, the solver cleanly untangles the equations without rounding errors and returns the 5 final, separated coefficients ($c_0 = 1176, c_1 = 3122$, etc.).

Once the coefficient vector C is recovered, the final integer product is computed by evaluating the product polynomial at the original radix: $W(B)$. In Python, this is implemented as base aligned large integer additions, allowing overlapping chunks to carry properly. For example, plugging our base $B = 100$ back into t, we force the coefficients into their proper place-value lanes. c_1 (3122) gets multiplied by 100^1, appending two zeros $(312, 200)$. We add the overlapping numbers to get our final result.

C++:

In contrast to Python's abstraction, the C++ implementation focuses on maximum efficiency and raw performance by using the GNU Multiple Precision Arithmetic Library (GMP) in order to showcase the speed of the algorithm.

Because C++ lacks native arbitrary-precision types, the mpz_class from GMP was used to handle integers that exceed 64-bit limits. This is essential for accuracy, as it provides exact integer arithmetic without the overflow risks of standard primitive types.

Unlike the generalized Python approach, the C++ code utilizes an optimized Toom-3 ($k=3$) logic. Instead of a runtime Gaussian solver, it employs Bodrato's Matrix, a hardcoded sequence of algebraic additions, subtractions, and divisions by 2 or 3 that are highly optimized for bit-shifting.

```
// 5. Interpolate (Bodrato's Matrix)
// With 5 points on the new curve, solve the linear equation using
// specific algebraic sequence detailed by Toom-cook
// Note: divisions by 2 and 3 are incredibly optimized and guarenteed to have no remainder
mpz_class r4 = r_inf;
mpz_class M1 = r1 - r0 - r4;
mpz_class Mm1 = r_m1 - r0 - r4;
mpz_class Mm2 = r_m2 - r0 - 16 * r4;

mpz_class r2 = (M1 + Mm1) / 2;
mpz_class A = M1 - r2;
mpz_class B = (4 * r2 - Mm2) / 2;
mpz_class r3 = (B - A) / 3;
mpz_class r1_final = A - r3;
```

To maximize speed, the C++ implementation uses a base-case threshold (e.g., 200 bits). For numbers below this size, the overhead of polynomial splitting exceeds the benefits, so the algorithm defaults to standard hardware multiplication.

By using GMP's C++ wrapper, the implementation manages memory overhead effectively, ensuring that the heavy recursive depth of the Toom-Cook algorithm does not lead to stack exhaustion or memory leaks.

Java:

The main difference between our Java implementation and the others was not due to implementation-based constraints, but rather a choice: while Java features a Big Integer library, we decided to implement our own.

Briefly we will discuss other implementation differences. The java implementation, like the C++ implementation, is specific to Toom-3. Also like the C++ implementation, it employs the Bodrato sequence, although not in matrix form (although the underlying math is the same). The Java implementation uses a base of 10^9 - while less efficient than a power of 2 base, this was far more feasible for implementing the custom Massive Integer class given the scope of the assignment.

The custom Massive Integer class features at its core an array of ints, stored in little-endian order (to make coding the inner arithmetic more intuitive), with a sign-magnitude bit as well. The modular login includes methods for addition, subtraction, schoolbook multiplication, as well as more specific operations like scalar multiplication, scalar division, and leftshift, implemented differently as these operations can be made computationally cheaper than the standard ones. There are other helpful methods, such as trim, which gets rid of leading 0's, or get magnitude. The Massive Integer Class is technically mutable; the sign-flip method does not create a new object, and multiple objects can point to the same int array. Perhaps this is bad practice, but the careful, modular way we coded our arithmetic ensures it is never an issue; all arithmetic operations create a new Massive Int object with new values. In the rare instance we need to flip the sign of a Massive Int outside of the class logic, we create a shallow copy, ensuring multiple pointers do not point to the same object. Since all arithmetic creates a new object, it is acceptable that our shallow copies point to the same int array.

Due to a combination of the way the Toom-3 algorithm was coded and the nature of the custom Massive Integer class, the Java implementation is likely the least flexible of the implementations; however, the custom Massive Integer class would work as well for any Toom-k algorithm of arbitrary k. And this inflexibility is not across the board - the Java version allows for easy reconfiguration of the threshold at which the recursive algorithm reaches the base case and switches to schoolbook multiplication.

Summary of Programming Challenge:

"Toom-any Cooks in the Kitchen" asks students to implement a fully generalized Toom-Cook-k multiplication algorithm, where k (the number of polynomial pieces, 2–8) is a runtime parameter rather than a fixed constant. Given two large integers A and B (up to 10,000 digits each), the program must output their product and the total count of recursive delegations made during the computation.

The learning objectives are for the student to understand why the algorithm is designed the way it is, as well as why people tend to use low k-values, with specific “magic numbers” that make solving the equations simpler. It also aims to allow students to understand the general solution that the magic numbers replace, by making them implement polynomial interpolation via Gaussian elimination on a Vandermonde system, with exact (rational/integer) arithmetic.

In order to show them why Toom-3 specifically is used, as well as understand the generalization of the algorithm further, we create a programming challenge where the student must implement a solution that can take in a variable “k”, the amount of pieces to split the integers into, and use that to complete the Toom-Cook multiplication.

Key Ideas for Solving Programming Challenge:

The main challenges can be split into two groups, the overall challenges when implementing any Toom-Cook, and the challenges that come from implementing it with a variable k. Some of the main challenges that arise from the problem in general are negative intermediate values, and exact interpolation. Evaluating polynomials at points like −1, −2 produces negative chunk evaluations. The interpolated product-polynomial coefficients can also be negative (even though the final integer product is non-negative). Students must use signed big-integer arithmetic throughout, which is easy to miss if they assume multiplication of non-negatives means all intermediates are non-negative.

Exact interpolation may be a challenge because floating-point arithmetic is insufficient for 10,000-digit inputs. The correct approach is either exact rational arithmetic (Python's Fraction), or careful integer arithmetic.

The implementation of a general solution carries a few additional challenges. Instead of being able to use a preset group of equations to perform interpolation, the student must create a general solution that is able to correctly solve the problem for any given k. They must also make sure that the threshold adapts to the value of k, so that they are able to stop recursing at reasonable values.

The time complexity of the solution is variable on the value of k, but in general the recurrence is $T(n) = (2k - 1) \cdot T(n/k) + O(k^2 * n)$, which by the master theorem gives $T(n) = O(n^{log_k(2k-1)})$. For k = 3 this is ≈ $O(n^{1.465})$, better than Karatsuba's $O(n^{1.585})$. For k = 2 it matches Karatsuba exactly $O(n^{1.585})$. The $O(k^2 * n)$ overhead at each level comes from

evaluating two degree-(k−1) polynomials at 2k−1 points and solving the 2k−1 × 2k−1 Vandermonde linear system, both of which are polynomial in k and linear in n.

Conclusion:

The Toom-Cook algorithm represents a significant step up from basic $O(n^2)$ multiplication, utilizing polynomial math to achieve an asymptotic complexity of $\Theta(n^{log(2k-1)/log(k)})$. Whether implemented through Python's flexible Fraction logic or C++'s high-performance GMP library, the algorithm demonstrates the power of combining algebraic structures with recursive design to solve fundamental computational problems.